

enable HSTS (HTTP Strict Transport Security) and marking all cookies with the 'secure' flag.

Apart from authenticating your web server, SSL/HTTPS will also provide security against man-in-the-middle attacks. It will also provide bi-directional encryption which safeguards your communication against eavesdropping, tampering and forging. An SSL certificate costs around ₹4,400 per year for a single domain and is fairly easy to setup on any server. Make sure that you redirect all your traffic on HTTP to HTTPS along with the HSTS header mentioned above along with 'secure' cookies, you can do this inside your server/middleware configuration.

XSS Protection

XSS attacks allow a user to inject client side scripts into the browsers of other users. This is achieved by storing malicious Javascript code snippets in the database. If the same data is fetched and displayed to other users which will cause that particular snippet to be executed in the user's browser. XSS attacks are usually achieved by the virtue of a single bad practice, not sufficiently sanitizing user generated data before including it in a page.

Sanitizing all user generated data is the most effective way of protecting yourself against such attacks. If you're using a templating system (use one if you're not already) to generate your web pages, then most of them have excellent in-built mechanisms which sanitize such data making sure that you're 95% of the way there. The key is to understand the limitations of such mechanisms and work your way around them, you'll have to be very careful when working with output other than HTML where you'll have an entirely separate characters and words which require escaping.

SQL Injection

The go to attack for demonstration, by ethical hacking workshops. Essentially, the attack is a result of a user being able to execute arbitrary SQL code on a database which can lead to records being leaked or deleted. If you're using some sort of ORM layer in your application, then for all practical intents and purposes you are protected from SQL injection.

It may be the case that your application requirements force you not to use such an ORM and require you to directly run native or raw SQL queries, therein as the bard would say, lies the rub. If native or raw SQL is not properly escaped before running then



Protection is absolutely essential

*w3af

(web application attack and audit framework)

<http://w3af.org>

>>You can easily find some of the most common vulnerabilities in your website using this nifty little tool. Written in Python, this cross-platform tool with it's 130+ plugins it is able to identify most of the common web application vulnerabilities like SQL injections, OS commanding, remote file inclusions, cross-site scripting (XSS), unsafe file uploads and so on.



there are all chances that someone can wreak havoc within your system. You must use native queries absolutely sparingly if at all you must. The key to protecting yourself from such attacks is to always be careful to properly escape any parameters that the user can control.

CSRF Protection

CSRF or Cross Site Request Forgery is when unauthorized commands from a malicious user are executed using the credentials of another user without that user's knowledge or consent. One can say that this attack is a mirror of an XSS attack, in case of XSS – it exploits the trust a user has for a particular site, CSRF exploits the trust that a site has in a user's browser.

There are a number of countermeasures to protect yourself from CSRF attacks something that you can do right off the bat is that you can embed a user-specific token in all form submissions and side-effect URLs prevents CSRF; the attacker's site cannot put the right token in its submissions. You must limit the lifetime of session cookies and make sure that the client provides authentication data in the same HTTP Request used to perform any operation with some security implications. A more straightforward and effective solution is to use a CSRF filter like the OWASP's CSRFGuard (https://www.owasp.org/index.php/Category:OWASP_CSRFGuard_Project | <http://goo.gl/FiwlPE>). The filter is then responsible for handling CSRF tokens in the request and response HTML.

Clickjacking

A very devious form of attack in which the a user is tricked into clicking something different from what the user perceives they're clicking, which in turn leads to revealing confidential information to the attacker or at times allow the attackers to take control of their system. It is carried out by a malicious site wrapping another site in a frame.

Standard protection against clickjacking is for the server to define a Content Security Policy. CSP defines a standard HTTP header which allows websites to declare approved sources of content that browsers should be allowed to load on that page. This covers JavaScript, CSS, HTML, frames, fonts, images and embeddable objects. The standard CSP header Content-Security-Policy is supported by Firefox 23 and Chrome 25 onwards. Another solution is to use the X-Frame-Options, which will force a compliant browser to deny any framing or prevent framing by external sites or allow framing only by specified site depending on the options in the header. >